

Fuzz & Furious: Speeding Up CAN Bus Fuzz Testing In Source Code Level

11 min read · Nov 5, 2023



Reza Zamiri

Follow



Listen



Share

... More

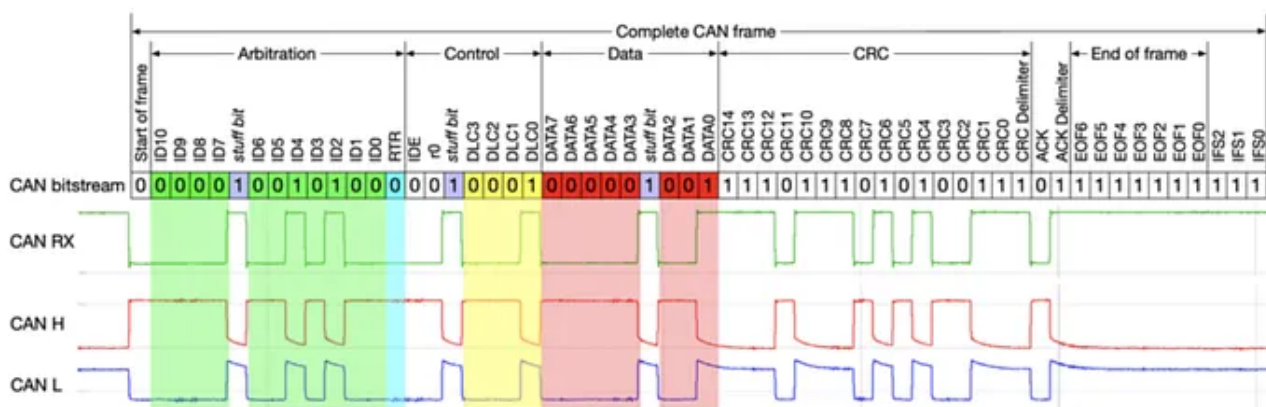
Introduction:

Today's vehicles are like high-tech machines, filled with Electronic Control Units (ECUs) and complex lines of code. Making sure these digital systems are secure is essential for the car's performance and safety.

In this blog post, we'll explore why security testing in the automotive industry matters so much. We'll also introduce you to a powerful technique called fuzz testing, which acts like a digital mechanic, carefully checking for hidden issues in automotive software. Then we will apply this method against CANBUS for finding a sample vulnerability.

Understanding the CAN bus

In the age of modern automotive systems, the Controller Area Network (CAN) bus protocol emerges as a vital thread that connects various electronic control units (ECUs) together. Serving as the nervous system of a vehicle, the CAN bus enables seamless communication among components, ranging from engine control units and sensors to infotainment systems and safety mechanisms.



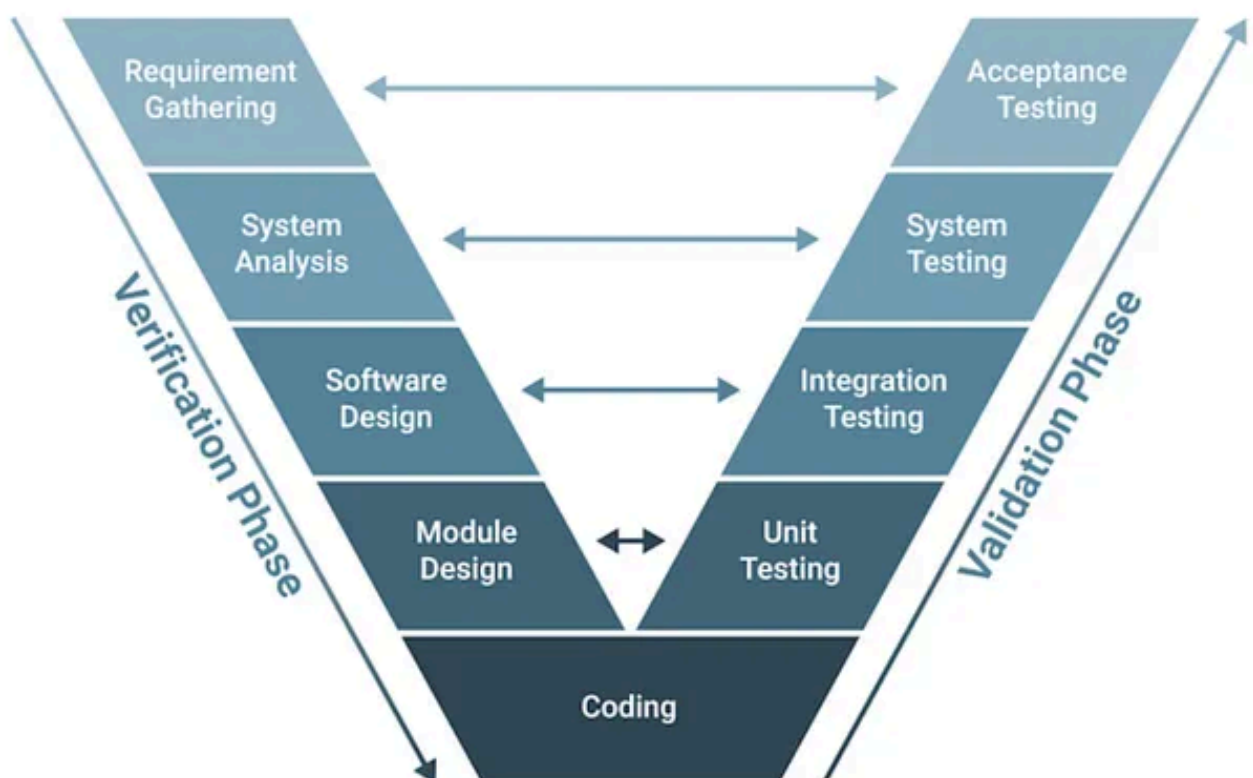
Imagine the symphony of activities occurring within a contemporary automobile. As you step on the accelerator, sensors gather data from various points within the vehicle: the engine's RPM, wheel rotation speed, throttle position, and much more. Simultaneously, these sensors transmit their readings to ECUs responsible for interpreting and acting on this data. This real-time information exchange is the lifeline of a modern car's functionality, and it's orchestrated by the CAN bus protocol.

Fuzz Testing

In the ever-evolving landscape of software security, one approach stands out for its effectiveness in exposing vulnerabilities and weaknesses: fuzz testing. Also known as fuzzing, this technique involves bombarding a software system with a wide range of inputs, often malformed or unexpected, to identify how the system under test will behave.

Fuzz testing and V-model

The V-model, often referred to as the "Verification and Validation Model", is an SDLC model where the execution of processes happens in a sequential manner in a V-shape. The V-Model demonstrates the relationships between each phase of the development life cycle and its associated phase of testing.



Fuzz testing is a valuable testing technique that can be applied at different stages of the V-model to uncover software vulnerabilities and enhance the quality and security of the developed software.

- **Unit-Level Testing:**

Unit testing focuses on testing individual components or units of code in isolation to ensure their correctness. Thus, fuzz testing at this stage is limited to whitebox testing at the source code level. Fuzzers like **Libfuzzer** or **AFL** are quite useful here.

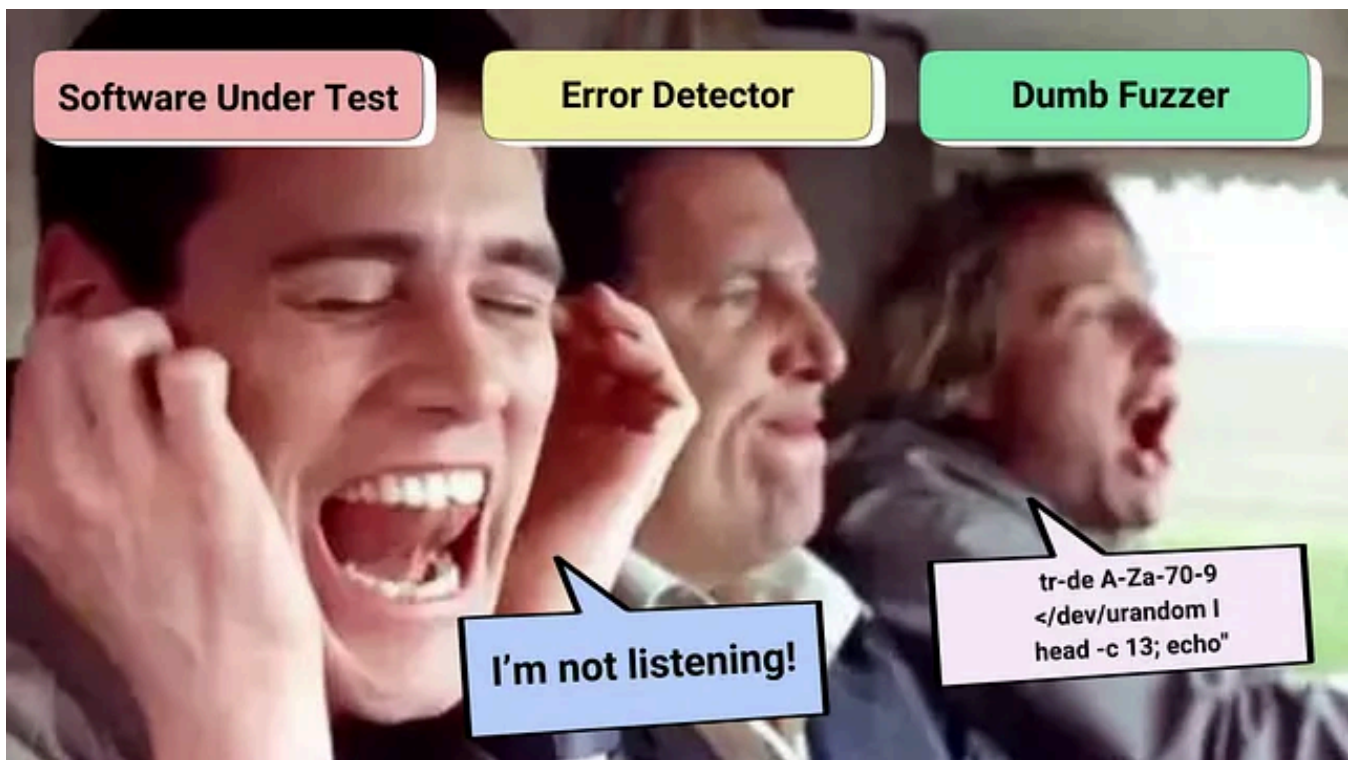
- **System-Level Testing:**

This phase involves testing the integrated system as a whole and assessing interactions between various components and subsystems. Fuzz testing at the system level aims to uncover vulnerabilities related to how the entire system handles unexpected or malicious inputs. CANBus fuzzers like **CaringCaribou** (open source) or **Defensics** (commercial) which use black box methodology are good examples.

Dumb vs Smart Fuzz testing

What happens when you try to open a pdf file in a media player? sounds ridiculous, that is why the dumb fuzz testing is ridiculous too. Every software component that accepts an input (through file, network, stdin, etc) has a grammar that defines the structure of that input, the non-standard inputs will not be processed by the software. In fuzz testing, it is important to make sure that our test cases are covering all branches of (abstract syntax tree) the software under test. For these reasons, the feedback-guided fuzzers were born to help the fuzz engines to generate smart test cases using the code coverage feedback. We will not get into details of the theory behind this approach, but highly recommend you to have a look at some resources.

Thus, the more random data you use to generate the test cases, the more dumb fuzz testing you will have.



Is fuzz testing enough?

Of course not! The goal of Fuzz testing is to make efficient inputs that can reach a vulnerable path in the software under test.

What happens if the test case hits the vulnerable part of the code? It depends! Sometimes it would be easy to monitor the situation of the software by looking at exit codes and operating system signals. A good example is the Linux SIGSEGV signal which is a signal in Unix-like operating systems (including Linux) that is generated when a program tries to read from or write to memory that is outside its allocated space.

However, there are vulnerabilities like reading from uninitialized memory which can lead to undefined behavior, but their consequences may not always result in immediate software crashes. Detecting and diagnosing such memory errors typically requires more advanced tools.

The art of memory error detection:

Memory error detection means identifying issues related to the allocation, use, and deallocation of memory in a software program (usually memory-unsafe languages like C/C++). These errors can include (but not limited to):

- Null Pointer Dereference: Attempting to access or modify data through a null pointer.

- **Buffer Overflows:** Writing or reading data outside the bounds of an allocated buffer, which can lead to data corruption or security vulnerabilities.
- **Use-After-Free:** Accessing memory that has been deallocated, potentially leading to unpredictable behavior.
- **Memory Leaks:** Failing to deallocate memory after it is no longer needed, causing the program to consume increasing amounts of memory over time.

Valgrind vs Sanitizers:

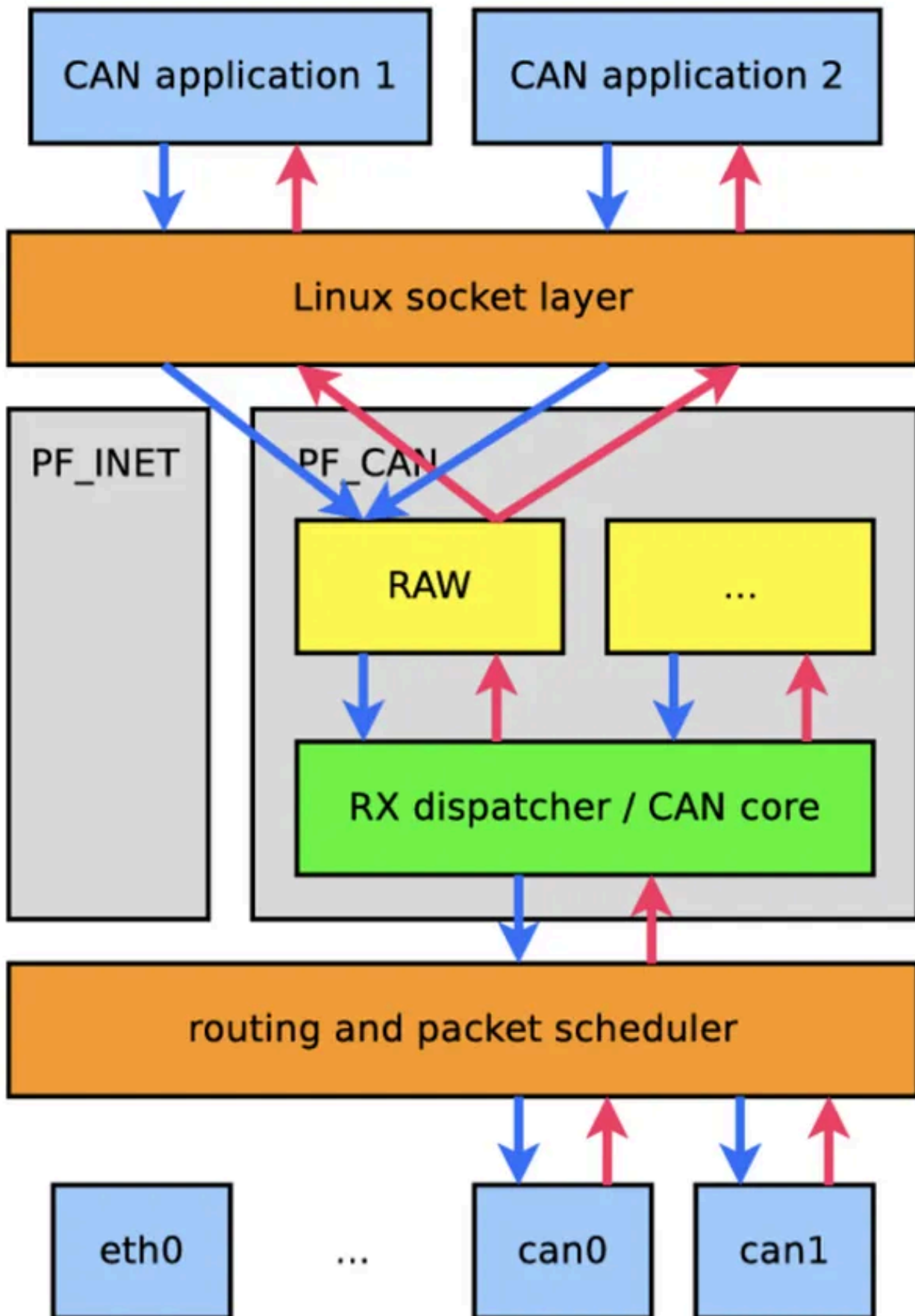
Valgrind and memory sanitizers are both powerful tools for detecting memory-related issues in software, but they have distinct characteristics and use cases. Valgrind, with its Memcheck tool, offers comprehensive dynamic analysis capabilities and can identify a wide range of memory errors, including invalid reads/writes and memory leaks. It operates as a separate runtime environment, which can be slower but provides detailed error reports. On the other hand, memory sanitizers like AddressSanitizer (ASan) and MemorySanitizer (MSan) offer faster analysis with minimal performance overhead by instrumenting the code during compilation. ASan is particularly effective at catching buffer overflows and use-after-free errors, while MSan specializes in detecting uninitialized memory reads. Choosing between Valgrind and sanitizers depends on specific requirements and constraints, with Valgrind offering extensive analysis and sanitizers providing efficient and targeted memory error detection.

Case Study: SocketCAN

As the CAN standard does not include common communication features, such as flow control, device addressing, and transportation of data blocks larger than one message, and above all, application data, many implementations of higher layer protocols were created. Several are standardized for a business area, although all can be extended by each manufacturer.

SocketCAN is a set of open-source CAN drivers and a networking stack contributed by Volkswagen Research to the Linux kernel. SocketCAN was formerly known as the Low Level CAN Framework (LLCF).

The SocketCAN concept extends the Berkeley sockets API in Linux by introducing a new protocol family, PF_CAN, that coexists with other protocol families, such as PF_INET for the Internet Protocol.



SocketCAN serves as a bridge between the CAN bus and user-level applications in a Linux environment. It exposes a set of socket-based APIs that allow applications to interact with CAN devices using standard socket operations.

In this case study, we'll walk through a simple implementation of a CAN receiver using SocketCAN.

Vulnerable CAN receiver:

Let's write a CAN receiver which contains a vulnerability with a specific condition. The code below will read any CAN messages from the bus and print the received CAN id, CAN dlc and data frame.

CanReceiver.c

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <sys/ioctl.h>
#include <net/if.h>
#include <linux/can.h>
#include <linux/can/raw.h>

// Function to process a received CAN frame
void processCANFrame(struct can_frame frame) {
    char buffer[6];
    printf("Received CAN frame:\n");
    printf("ID: 0x%03X\n", frame.can_id);
    printf("DLC: %d\n", frame.can_dlc);
    printf("Data: ");
    for (int i = 0; i < frame.can_dlc; ++i) {
        printf("%02X ", frame.data[i]);
    }
    printf("\n");

    for (int i = 0; i < frame.can_dlc; ++i) {
        // Check for specific conditions
        if (frame.can_id == 0x123 && frame.can_dlc == 8 &&
            frame.data[0] == 0xAA && frame.data[1] == 0xBB &&
            frame.data[2] == 0xCC && frame.data[3] == 0xDD &&
            frame.data[4] == 0xEE && frame.data[5] == 0xFF &&
            frame.data[6] == 0x11 && frame.data[7] == 0x12) {
            printf("%02X ", frame.data[i]);
            memcpy(buffer, frame.data, frame.can_dlc);
        }
    }
}

int main() {

    int s;
    struct sockaddr_can addr;
```



```

struct ifreq ifr;
// Create socket
s = socket(PF_CAN, SOCK_RAW, CAN_RAW);
if (s == -1) {
perror("Socket creation error");
return -1;
}
// Specify CAN interface (e.g., can0)
strcpy(ifr.ifr_name, "can0");
ioctl(s, SIOCGIFINDEX, &ifr);
// Configure socket address
addr.can_family = AF_CAN;
addr.can_ifindex = ifr.ifr_ifindex;
// Bind socket to the CAN interface
if (bind(s, (struct sockaddr*)&addr, sizeof(addr)) == -1) {
perror("Socket binding error");
close(s);
return -1;
}
while (1) {
struct can_frame frame;
ssize_t nbytes = read(s, &frame, sizeof(struct can_frame));
if (nbytes < 0) {
perror("Socket read error");
break;
}
// Process received CAN frame
processCANFrame(frame);
}
close(s);
return 0;
}

```

This code also contains a classic buffer overflow vulnerability in the ProcessCANframe function when it copies the data from a CAN message into a limited memory size.



Before we run the code, we need to setup a virtual can interface using the following commands:

```
sudo ip link add dev can0 type vcan
sudo ifconfig can0 up
```

Now just compile the code with clang and run it, the program will wait for an upcoming CAN message to print its content:

```
clang Can2.c -o Can2
```

Now it's time to send a random CAN message, I used one of the tools from **Canutils** called Cansend, we just need to specify canid and arbitrary data. An example of sending a message to canid 123:

```
(kali@kali)-[~/CAN/CanFunction]
$ cansend can0 123#AABBCCDDEEFF1111

(kali@kali)-[~/CAN/CanFunction]
$
```

And the program after reading the message:

```
(kali㉿kali)-[~/CAN/CanFunction]
$ ./Can2
Received CAN frame:
ID: 0x123
DLC: 8
Data: AA BB CC DD EE FF 11 11
█
```

Everything is fine! Now what happens when we change the data to: AA BB CC DD EE FF 11 12?

```
(kali㉿kali)-[~/CAN/CanFunction]
$ ./Can2
Received CAN frame:
ID: 0x123
DLC: 8
Data: AA BB CC DD EE FF 11 11
Received CAN frame:
ID: 0x123
DLC: 8
Data: AA BB CC DD EE FF 11 12
zsh: segmentation fault ./Can2

(kali㉿kali)-[~/CAN/CanFunction]
$
```

Segmentation fault! Now the thing is how we can find this vulnerability by automated testing(Fuzz)? To show the difference between the dumb and smart fuzz testing, we work on two approaches. The first one consists of instrumenting our software under test with a memory error detector(Asan) and then starting the fuzz testing using a very simple (semi)dumb fuzzer.

For the second part, we use a code coverage guided fuzzer(Libfuzzer) and also use the Asan as the error detector.

Semi-dumb fuzz testing:

I call this approach semi-dumb because we actually are helping the fuzzer to generate some valid test cases and they are not completely random characters. We use the same structure of the can frame and only ask our semi-dumb fuzzer to generate random CAN message data.

Semi-dumb.c

```
srand(time(NULL));  
...  
while (1) {  
    frame.can_id = 0x123;  
    frame.can_dlc = 8;  
    for (int i = 0; i < frame.can_dlc; i++) {  
        frame.data[i] = rand() & 0xFF; // Generate random data byte (0x00 to 0xFF)  
    }  
    if (write(s, &frame, sizeof(struct can_frame)) != sizeof(struct can_frame)) {  
        perror("Write");  
        return 1;  
    }  
}
```

In the next step, I use the Clang compiler to help me instrument my vulnerable code with Asan, just by using the following commands:

```
clang -fsanitize=address CanReceiver.c -o Fuzz
```

Now let's run it against our CAN receiver and check the result. No crash after a couple of hours(and probably after a couple of days!)

```
Received CAN frame:
ID: 0x123
DLC: 8
Data: A3 F5 38 63 8B 27 6D EB
Received CAN frame:
ID: 0x123
DLC: 8
Data: E1 51 78 54 29 08 E5 50
Received CAN frame:
ID: 0x123
DLC: 8
Data: E3 61 3B 2D C0 C3 24 EC
Received CAN frame:
ID: 0x123
DLC: 8
Data: EA 1B 01 C3 27 9E 4F CA
Received CAN frame:
ID: 0x123
DLC: 8
Data: 93 88 2D 1F AF 9B 0A 91
Received CAN frame:
ID: 0x123
DLC: 8
Data: EC 82 E5 16 8A CA 66 6D
Received CAN frame:
ID: 0x123
DLC: 8
Data: 2B A1 9B EB 64 BF D7 4F
Received CAN frame:
ID: 0x123
DLC: 8
Data: DA D8 12 01 77 62 CB 0A
Received CAN frame:
ID: 0x123
DLC: 8
Data: EA F8 29 99 93 33 2A 80
^C

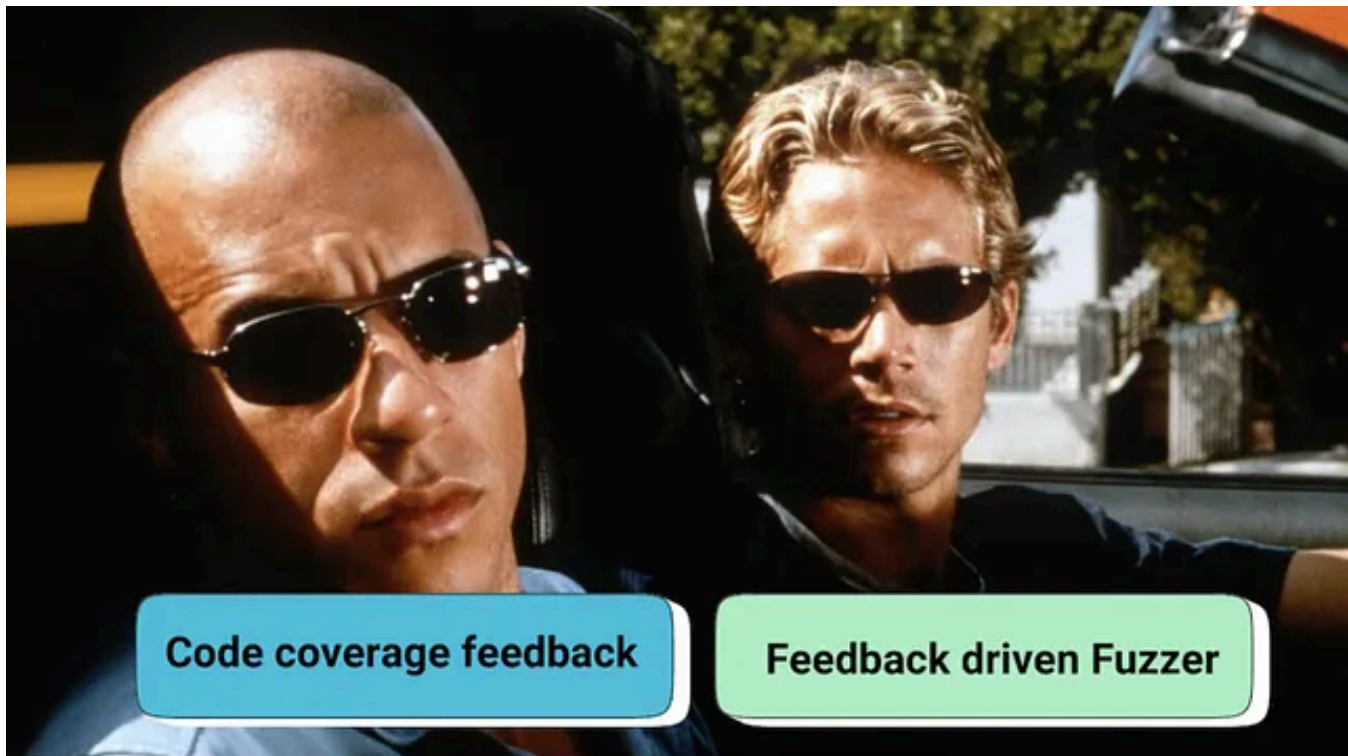
(kali@kali)-[~/CAN/CanFunction]
$
```

LibFuzzer: The Fuzzing Powerhouse

libFuzzer is a coverage-guided fuzz testing engine. Developed as part of LLVM, is designed to be highly efficient, effective, and easy to use. Its primary focus is on increasing code coverage during fuzz testing, which means exploring as many paths in the codebase as possible. This coverage-guided approach improves the likelihood of finding hidden bugs and vulnerabilities that might remain undetected using other methods.

Embracing Coverage-Guided Fuzzing

At the heart of libFuzzer's effectiveness lies its focus on coverage-guided fuzzing. This innovative approach harnesses the power of code coverage analysis to drive the fuzzing process. The underlying principle is simple yet profound: the more code paths a fuzzer explores, the higher the chances of discovering hidden vulnerabilities.



Writing the Fuzz target:

Now let's use the magic power of coverage-guided fuzz testing by writing a fuzz target for our vulnerable function:

FuzzTarget.c

```
#include <stdint.h>
#include <stdio.h>
#include <string.h>
// Define a structure to represent a CAN frame
struct can_frame {
    int can_id;
    int can_dlc;
    uint8_t data[8];
};
// Define the LLVMFuzzerTestOneInput function, which is the entry point for the fuzzer
int LLVMFuzzerTestOneInput(const uint8_t *data, size_t size) {
    // Create a CAN frame structure to hold the data
    struct can_frame frame;
    if (size < sizeof(frame)) {
        return 0;
    }
    // Set the CAN frame's identifier and data length code
    frame.can_id = 0x123;
    frame.can_dlc = 8;
    // Copy the input data into the CAN frame's data payload
    memcpy(frame.data, data, sizeof(frame.data));
    // Call the processCANFrame function to handle the CAN frame
    processCANFrame(frame);
    return 0;
}
```

Don't forget to include the vulnerable software unit (which is our function under test)

Function.c

```
// Function to process a received CAN frame
void processCANFrame(struct can_frame frame) {
    char buffer[6];
    printf("Received CAN frame:\n");
    printf("ID: 0x%03X\n", frame.can_id);
    printf("DLC: %d\n", frame.can_dlc);
    printf("Data: ");
    for (int i = 0; i < frame.can_dlc; ++i) {
        printf("%02X ", frame.data[i]);
    }
    printf("\n");

    for (int i = 0; i < frame.can_dlc; ++i) {
```

```
// Check for specific conditions
if (frame.can_id == 0x123 && frame.can_dlc == 8 &&
    frame.data[0] == 0xAA && frame.data[1] == 0xBB &&
    frame.data[2] == 0xCC && frame.data[3] == 0xDD &&
    frame.data[4] == 0xEE && frame.data[5] == 0xFF &&
    frame.data[6] == 0x11 && frame.data[7] == 0x12) {
    memcpy(buffer, frame.data, frame.can_dlc);
}
}
}
```

Now just compile the codes with the following command:

```
clang -fsanitize=address, fuzzer FuzzTarget.c Function.c -o Fuzz
```

Just run the binary and wait for some **minutes** only:

```
ID: 0x123
DLC: 8
Data: AA BB CC DD EE FF 11 12
=====
**2225**ERROR: AddressSanitizer: stack-buffer-overflow on address 0x7ffc83cb3d07 at pc 0x55adf3e012ca bp 0x7ffc83cb3cb0 sp 0x7ffc83cb3480
WRITE of size 8 at 0x7ffc83cb3d07 thread T0
#0 0x55adf3e012c9 in __asan_memcpy (/home/kali/CAN/new/Fuzz+0xde2c9) (BuildId: 18021b4f55762c796baf19dd3d4206c0e03850a2)
#1 0x55adf3e3d668 in processCANFrame (/home/kali/CAN/new/Fuzz+0x11a668) (BuildId: 18021b4f55762c796baf19dd3d4206c0e03850a2)
#2 0x55adf3e3db1e in LLVMFuzzerTestOneInput (/home/kali/CAN/new/Fuzz+0x11ab1e) (BuildId: 18021b4f55762c796baf19dd3d4206c0e03850a2)
#3 0x55adf3d5533 in fuzzer::Fuzzer::ExecuteCallback(unsigned char const*, unsigned long) (/home/kali/CAN/new/Fuzz+0x42533) (BuildId: 18021b4f55762c796baf19dd3d4206c0e03850a2)
#4 0x55adf3d64c59 in fuzzer::Fuzzer::RunOne(unsigned char const*, unsigned long, bool, fuzzer::InputInfo*, bool, bool*) (/home/kali/CAN/new/Fuzz+0x41c59) (BuildId: 18021b4f55762c796baf19dd3d4206c0e03850a2)
#5 0x55adf3d663b9 in fuzzer::Fuzzer::MutateAndTestOne() (/home/kali/CAN/new/Fuzz+0x433b9) (BuildId: 18021b4f55762c796baf19dd3d4206c0e03850a2)
#6 0x55adf3d66f35 in fuzzer::Fuzzer::Loop(std::vector<fuzzer::SizedFile, std::allocator<fuzzer::SizedFile> >6) (/home/kali/CAN/new/Fuzz+0x43f35) (BuildId: 18021b4f55762c796baf19dd3d4206c0e03850a2)
#7 0x55adf3d55172 in fuzzer::FuzzerDriver(int*, char***, int (*)(unsigned char const*, unsigned long)) (/home/kali/CAN/new/Fuzz+0x32172) (BuildId: 18021b4f55762c796baf19dd3d4206c0e03850a2)
#8 0x55adf3d7f162 in main (/home/kali/CAN/new/Fuzz+0x5c162) (BuildId: 18021b4f55762c796baf19dd3d4206c0e03850a2)
#9 0x7fb00c967189 (/lib/x86_64-linux-gnu/libc.so.6+0x27189) (BuildId: b1c6521cd0ab872b70c21377654e966ea9d438c9)
#10 0x7fb00c967244 in __libc_start_main (/lib/x86_64-linux-gnu/libc.so.6+0x27244) (BuildId: b1c6521cd0ab872b70c21377654e966ea9d438c9)
#11 0x55adf3d49990 in _start (/home/kali/CAN/new/Fuzz+0x26990) (BuildId: 18021b4f55762c796baf19dd3d4206c0e03850a2)

Address 0x7ffc83cb3d07 is located in stack of thread T0 at offset 71 in frame
#0 0x55adf3e3cc2f in processCANFrame (/home/kali/CAN/new/Fuzz+0x119c2f) (BuildId: 18021b4f55762c796baf19dd3d4206c0e03850a2)

This frame has 2 object(s):
[32, 48) 'frame'
[64, 71) 'buffer' <== Memory access at offset 71 overflows this variable
HINT: this may be a false positive if your program uses some custom stack unwind mechanism, swapcontext or vfork
(longjmp and C++ exceptions *are* supported)
SUMMARY: AddressSanitizer: stack-buffer-overflow (/home/kali/CAN/new/Fuzz+0xde2c9) (BuildId: 18021b4f55762c796baf19dd3d4206c0e03850a2) in __asan_memcpy
Shadow bytes around the buggy address:
0x10001078e750: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x10001078e760: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x10001078e770: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x10001078e780: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x10001078e790: 00 00 00 00 00 00 00 00 f1 f1 f1 f1 00 00 f2 f2
=>0x10001078e7a0:[07]f3 f3 f3 00 00 00 00 00 00 00 00 00 00 00 00
0x10001078e7b0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x10001078e7c0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x10001078e7d0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
0x10001078e7e0: f1 f1 f1 f1 00 00 f3 f3 00 00 00 00 00 00 00
0x10001078e7f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
Shadow byte legend (one shadow byte represents 8 application bytes):
Addressable: 00
Partially addressable: 01 02 03 04 05 06 07
```



```

Shadow byte legend (one shadow byte represents 8 application bytes):
Addressable: 00
Partially addressable: 01 02 03 04 05 06 07
Heap left redzone: fa
Freed heap region: fd
Stack left redzone: f1
Stack mid redzone: f2
Stack right redzone: f3
Stack after return: f5
Stack use after scope: f8
Global redzone: f9
Global init order: f6
Poisoned by user: f7
Container overflow: fc
Array cookie: ac
Intra object redzone: bb
ASan internal: fe
Left alloca redzone: ca
Right alloca redzone: cb
==2225==ABORTING
MS: 3 ShuffleBytes-ChangeBinInt-CMP- DE: "\022\000\000\000\000\000\000\000"-; base unit: c008106fc53f5fa90786633d1f74036328303992
0xaa,0xbb,0xcc,0xdd,0xee,0xff,0x11,0x12,0x0,0x0,0x0,0x0,0x0,0x0,0xed,
\252\273\314\335\356\377\021\022\000\000\000\000\000\000\000\355
artifact_prefix='./'; Test unit written to ./crash-a5b286ee45171773628e962711593718532004e0
Base64: qrvM3e7/ERIAAAAAAAAA7Q==
(kali@kali)-[~/CAN/new]
$

```

Demo Video:

[\(9\) DEMO | Fuzz & Furious: Speeding Up CAN Bus Fuzz Testing In Source Code Level — YouTube](#)

Conclusion:

In this post, we’ve witnessed the critical importance of fuzz testing in enhancing automotive software security. From the intricacies of smart fuzz testing to the integration of V-model principles, we’ve explored a comprehensive approach to security testing. The SocketCAN case study and the distinction between dumb and smart fuzz testing have provided valuable insights. Our discussion on memory error detection tools like AddressSanitizer and the power of libFuzzer’s coverage-guided testing has equipped us to navigate the complex landscape of automotive cybersecurity. As we continue on this ever-evolving road, these robust testing methodologies serve as our compass, ensuring the safety and resilience of connected vehicles in an increasingly interconnected world.